Ledger-Free Trustless Battleship:

Self-Contained Zero-Knowledge Shot Proofs with Public-History Binding

Hadi Ghazi
hadigghazi@gmail.com

July 2026

Abstract

Zero-knowledge proofs let players of imperfect-information games answer questions about hidden state — "hit or miss?" — without revealing that state and without being able to lie. Every deployed zero-knowledge Battleship system we are aware of anchors the *public* game state (the shot history, the turn order, the win condition) on a blockchain, inheriting its latency, cost, and infrastructure. We show that the obvious ledger-free port of these protocols is unsound: if the prover supplies the shot history that "sunk" and "you win" are computed from, a losing player can misreport sunk ships and postpone provable defeat indefinitely, while producing proofs that verify. We give a simple fix with a useful general shape: every per-shot proof echoes the complete public shot history into its public output, and the verifier — who fired exactly those shots — checks it against its own record. Because in a two-player turn-based game each verifier locally possesses the canonical public state it needs, no consensus layer is required. Sunk detection and the win condition are computed inside the proof from the bound history, so match transcripts become independently auditable end-to-end, including termination. We implement the protocol on the RISC Zero zkVM as an open-source, peer-to-peer system — a local prover agent and browser interface per player, connected by an untrusted message relay — with a Blum coin flip for the first move, compile-time–disabled mock proving, and guest programs hardened to witness-independent instruction traces. On a commodity laptop CPU, a shot proof takes 30–47 s to produce and ≈30 ms to verify, making the system playable at a relaxed turn-based pace with no hardware acceleration and no infrastructure beyond a dumb message pipe.

1 Introduction

Battleship is played on trust. Each player privately places a fleet on a 10×10 grid; opponents call shots and are told "hit" or "miss" by the only party able to see the board. Nothing in the rules prevents a player from quietly relocating ships mid-game or denying that a ship was sunk; the game works between friends and fails between adversaries.

Cryptography removes the trust assumption. A player can *commit* to a board before the game and then, for every shot, produce a *zero-knowledge proof* that the answer is consistent with the committed board — revealing nothing else about it. This commit-and-prove pattern is folklore by now and has been implemented for Battleship several times: with hand-written arithmetic circuits verified by an Ethereum contract [12, 13], with a zkVM posting proofs to the NEAR blockchain [14], and as demonstration applications on privacy-oriented chains [15, 16]. The pattern of hiding game state behind commitments and proving actions against it also powers the on-chain game Dark Forest [17].

All of these systems share one structural decision: the *public* part of the game state — which cells have been fired at, whose turn it is, how many hits have landed, and therefore who has won — lives on a blockchain, and a smart contract acts as referee for everything that is not secret. The chain contributes no secrecy (commitments and proofs do that); what it contributes

is an *authoritative public history*. That contribution is easy to overlook, and removing the chain without replacing it breaks the protocol in a way that has nothing to do with the zero-knowledge machinery (Section 4): the statements "this shot sank a ship" and "I have lost" are functions of the shot history, and if the prover chooses the history, those statements become unfalsifiable. A losing player can produce perfectly valid proofs forever and simply never be provably defeated — we call this the *immortal fleet*.

This paper develops and evaluates a Battleship protocol with no ledger, no referee, and no trusted party of any kind, in which the complete game — including its *termination* — is verifiable. The technical core is a modest but, to our knowledge, previously unexploited observation:

> *In a two-player turn-based game, the public state a verifier needs in order to check a proof about an opponent's hidden state is exactly the sequence of the verifier's own past moves — which the verifier already knows. Binding that sequence into each proof's public output replaces the ledger.*

Concretely, every shot proof in our protocol takes the full public history of shots fired at the responding player as input, recomputes the commitment from the private board (locking the player to one board for the whole game), computes hit/miss, sunk, and *defeated* from board ∪ history inside the proof, and echoes the history into the proof's public journal. The verifier accepts only if the echoed history equals, element for element, the shots it actually fired. Each receipt is therefore *self-contained*: a third party given the two commitment receipts and the ordered shot receipts can re-verify an entire match, including who won, with no other information — the history fields chain the receipts to one another (Section 5.3).

Contributions.

1. We identify the *public-history binding problem* for ledger-free commit-and-prove games and give concrete attacks on the natural chain-free protocol, including sunk-report forgery and indefinite defeat postponement (Section 4).
2. We specify a complete two-party protocol — board commitment with an in-proof legality check, a Blum commit-reveal coin flip [4] for the first move, history-bound shot proofs, and an in-proof win condition — and analyse its security properties and their reductions (Sections 5–6).
3. We implement the protocol on the RISC Zero zkVM [7] in a deliberately trust-minimised deployment: each player runs a local prover agent and browser UI; an untrusted stateless relay forwards only public messages; mock proving is disabled at compile time; and all code that touches the secret board executes a fixed-shape instruction trace so that proving time does not leak board information (Section 7). The implementation is open source.
4. We evaluate on a commodity laptop CPU: proving and verification times, receipt sizes (composite and succinct), the effect of history length, and an empirical check of witness-independence (Section 8).

We stress what is *not* claimed. The commit-and-prove structure for Battleship is not new; our contribution is removing the ledger while keeping — and extending, via proven termination — the integrity guarantees, plus an engineering-complete artifact and measurements. Zero-knowledge proofs also cannot force a player to keep playing; aborts remain possible, as in every prior system, and we discuss the boundary in Section 9.

2 Background

Commitments. A commitment scheme lets a party fix a value v by publishing $c = \mathsf{Com}(v; r)$ such that c reveals nothing about v (*hiding*) and cannot later be opened to $v' \neq v$ (*binding*).

We use the standard salted-hash commitment $C = \text{SHA-256}(\mathsf{encode}(B) \,\|\, s)$ over a canonical board encoding and a fresh 256-bit salt s; binding follows from collision resistance, hiding from the unpredictability of the salted preimage (a random-oracle–style assumption standard for this construction).

Zero-knowledge proofs and zkVMs. A zero-knowledge proof [1] convinces a verifier that a statement is true while revealing nothing beyond its truth; proof systems with succinct, non-interactive proofs [6, 5] make the pattern practical for applications. A *zkVM* generalises circuit-level proving to whole programs: the prover executes a program on private inputs and emits a proof of correct execution. RISC Zero [7] proves execution of RISC-V binaries with a STARK [5]; a proving run yields a *receipt* containing a *journal* (the public outputs the program chose to commit) and a *seal* (the proof). Verification checks the seal against an *image ID* — a digest of the exact guest binary — so a verifying party learns that *that specific program* ran and produced *that journal*, and nothing else. Comparable systems include SP1 [8] and Jolt [9]; surveys and performance studies of the zkVM landscape appear in [10, 11].

Battleship. Two players secretly place one ship each of lengths $5, 4, 3, 3, 2$ (17 cells) on a 10×10 grid; ships are straight, axis-aligned, in bounds, and non-overlapping (touching is allowed). Players alternate firing at cells of the opponent's grid and are told hit or miss; when all cells of a ship have been hit it is announced sunk; a player whose 17 ship cells are all hit loses. We additionally impose the (standard-practice) rule that each cell may be targeted at most once per board, which bounds a game at 100 shots per side and, in our protocol, keeps both parties' history sequences in lockstep.

3 Related work

Cryptographic protocols for games predate practical ZK proofs: mental poker [2] dealt cards between mutually distrusting parties, and the completeness theorems for secure computation [3] imply in principle that *any* game can be played fairly. What has changed is practicality: succinct proof systems and zkVMs make per-move proving affordable.

On-chain ZK Battleship. BattleZips [12] (circom/Groth16) proves board legality at placement and hit/miss per shot; an Ethereum contract enforces turn order, stores declared shots, counts the 17 hits that end the game, and is the root of trust for all public state. The MIT 6.857 project [13] (circom/snarkjs on Ethereum) proves that each reported hit/miss is consistent with the hashed board; notably it does *not* prove board legality at placement — a player can commit a malformed board — and the win condition is again contract-side hit counting. RISC Zero's own example [14] moved the circuits into a zkVM with game state in a NEAR contract; it is unmaintained and predates modern receipt formats. Aleo and Mina host similar demonstrations [15, 16]. In all of these, removing the chain removes the component that fixes the public history; this paper is about what must replace it.

Off-chain ZK gaming. Dark Forest [17] pioneered commitment-hidden state in a deployed game, still with an on-chain core. State-channel constructions [19, 18] move interaction off-chain between on-chain checkpoints, and "ZK state channels" have been sketched for turn-based games; they still assume a chain for disputes and settlement. Our setting is stricter — there is no chain at any layer — and our observation is that for two-player turn-based games none is needed for *integrity*, only for remedies against abort (Section 9).

Fair coin flipping. We use Blum's classic commit-reveal protocol [4] to choose the first mover; its known limitation (the last revealer may abort on an unfavourable outcome) is subsumed by the abort non-guarantee that any ZK game protocol already carries.

4 The public-history binding problem

Consider the natural ledger-free protocol, which is also (up to notation) the protocol implemented by the systems above with the contract deleted. Let player P hold board B and salt s with published commitment $C = H(\mathsf{encode}(B)\|s)$.

- **Setup proof.** P proves knowledge of a *legal* B and s with $H(\mathsf{encode}(B)\|s) = C$; the journal is C.
- **Naive shot proof.** On shot x, P proves: "for the B, s behind C: $\mathit{hit} = \mathsf{occ}(B, x)$, and $\mathit{sunk} = \mathsf{sunk}(B, x, \mathcal{H})$" where $\mathcal{H}$ is the set of shots previously fired at P — *supplied by P as a private input.* The journal is $(C, x, \mathit{hit}, \mathit{sunk})$.

The hit bit is sound: it depends only on B and x, both bound by the journal. Everything that depends on $\mathcal{H}$ is not, because the verifier has no way to know which $\mathcal{H}$ the statement was evaluated on. Three concrete attacks follow.

A1: Sunk suppression. On the shot that completes a ship, a malicious responder proves against $\mathcal{H}' \subsetneq \mathcal{H}$ (e.g. omitting one earlier hit). The proof verifies, reports $\mathit{hit} = \mathsf{true}$, $\mathit{sunk} = \bot$, and the attacker's fleet status is misrepresented. In fleet variants where sunk announcements drive strategy, this is a material advantage; the attack cost is zero.

A2: Phantom sunk. Symmetrically, the responder pads $\mathcal{H}'$ with shots never fired, causing a ship to be reported sunk early — useful for feigning weakness, or for confusing opponents that track sunk counts.

A3: The immortal fleet. The win condition is $\mathsf{allsunk}(B, \mathcal{H})$. Whatever party computes it needs $\mathcal{H}$; without a ledger, implementations must either trust the loser's client to announce defeat (self-reporting) or have the winner's client count sunk reports — which A1 already corrupts. A losing player therefore never becomes *provably* defeated: they can answer every shot with a valid proof, suppress the final sunk reports, and the game never ends. No cryptographic assumption is violated; the statements being proven are simply about a history nobody agreed on. Notably, the pre-revision version of our own implementation had exactly this flaw — defeat was a self-reported boolean computed outside any proof — which is what prompted this analysis.

On-chain systems do not exhibit A1–A3 because the contract is the history: shots are transactions, and hit counting happens in consensus. The question is what replaces the contract when there is no chain.

The observation. In a two-player turn-based game, the history $\mathcal{H}$ relevant to proofs about P's board is precisely the sequence of shots *fired by the opponent V* — and V, the verifier of those proofs, chose and remembers every element of it. The canonical public state is already local to the party that needs it; it requires binding, not consensus. This suggests the fix: make $\mathcal{H}$ part of the *statement* rather than the witness, by echoing it into the journal, and let V compare it with its own record.

5 The protocol

We now specify the protocol as implemented. H is SHA-256. $\mathsf{encode}(B)$ lists the five ships sorted by a canonical ship id, each as (id, row, col, orientation), 4 bytes per ship, followed by the 32-byte salt; commitments are therefore well-defined independent of placement listing order.

5.1 Setup

Each player $P \in \{A, B\}$:

1. chooses a board B_P (interactively, in the UI) and samples $s_P \leftarrow \{0,1\}^{256}$ from the OS CSPRNG;
2. runs the commit guest, which *asserts* $\mathsf{legal}(B_P)$ (composition, bounds, non-overlap) and outputs journal $C_P = H(\mathsf{encode}(B_P) \| s_P)$, yielding receipt R_P^{com};
3. samples a coin nonce $n_P \leftarrow \{0,1\}^{256}$ and sends $(C_P, R_P^{\mathsf{com}}, H(n_P))$ to the opponent;
4. on receiving the opponent's tuple: verifies R^{com} against the pinned commit image ID, checks the journal equals the claimed C, and rejects a mirrored commitment $C = C_P$ (a copied commitment can never be answered — the copier lacks the witness — so games against it could only stall);
5. reveals n_P; verifies the opponent's reveal against $H(n)$; the first mover is decided by the XOR parity of the two nonces [4].

An illegal board thus never enters a game: there is no receipt for it. Note the contrast with [13], where placement legality is unproven.

5.2 Play

Let $F_V = (x_1, \ldots, x_k)$ be the ordered shots V has fired at P so far (the *fired list*), maintained by V's own agent. Shots must be pairwise distinct; P's agent refuses to answer a repeated cell.

On V's turn, V sends a shot x. P runs the shot guest on private input (B_P, s_P) and public input $(C_P, x, \mathcal{H})$ where $\mathcal{H} = (x_1, \ldots, x_k)$ is P's record of shots received. The guest:

1. recomputes $H(\mathsf{encode}(B_P) \| s_P)$ and *asserts* it equals C_P (board binding)
2. computes $\mathit{hit} = \mathsf{occ}(B_P, x)$; $\mathit{sunk} = \mathsf{sunkship}(B_P, x, \mathcal{H})$; $\mathit{def} = \mathsf{allsunk}(B_P, \mathcal{H} \cup \{x\})$
3. commits journal $J = (C_P,\ x,\ \mathit{hit},\ \mathit{sunk},\ \mathit{def},\ \mathcal{H})$ (history binding)

and returns the receipt. V accepts iff:

$$\mathsf{Verify}(R, \mathsf{id}_{\mathsf{shot}}) \ \wedge\ J.C = C_P \ \wedge\ J.x = x \ \wedge\ J.\mathcal{H} = F_V,$$

where the last check is *sequence* equality against V's own fired list, and only then appends x to F_V. If $J.\mathit{def} = \mathsf{true}$, V has won and holds a transferable proof of it; P's own agent reports the same bit to P from the same journal, so defeat is simultaneously undeniable and unfakeable. Roles then swap.

Because both the responder's history record and the verifier's fired list grow by exactly the accepted shot, the two stay equal by induction; any divergence — a dropped, injected, reordered, or replayed message, whether by the network, the relay, or a cheating peer — makes the very next verification fail closed.

5.3 Public auditability of transcripts

Define a match transcript as the two commitment receipts plus, per direction, the ordered sequence of shot receipts. A third party with no involvement in the game can check, using only the pinned image IDs: (i) every receipt verifies; (ii) within a direction, receipt k's journal history equals the shots of receipts $1..k{-}1$ (the receipts chain themselves — no signatures or timestamps are needed); (iii) the last receipt of the losing direction carries $\mathit{def} = \mathsf{true}$. Termination is thus not a claim by either player but a property of the transcript. This is the ledger-free analogue of "the contract says the game ended", and it is what the naive protocol cannot offer.

5.4 Cost

Per shot: one proof, one verification, and a journal of $O(|\mathcal{H}|)$ shot entries (2 bytes each before serialization). Over a maximal 100-shot direction the transcript's journal overhead is

$O(n^2) \approx 10^4$ cells' worth of bytes — negligible against the seals themselves (Section 8). A running-hash binding ($h_k = H(h_{k-1} \| x_k)$) would make journals $O(1)$ at the cost of requiring auditors to recompute the chain; with $n \leq 100$ the plain list is simpler and self-describing.

6 Security analysis

The adversary is a computationally bounded player who may deviate arbitrarily: choose any messages, any boards, any histories, abort at will. The relay is untrusted for everything except message delivery (it can observe and drop, but public messages are all it ever carries). We assume the collision resistance of SHA-256, the soundness and zero-knowledge of the RISC Zero STARK [7, 5], and correctness of the ∼300-line shared rules crate, which is compiled into the guest and is the protocol's entire trusted rulebook (both players pin its image IDs and every verification checks them).

Claim 1 (Response soundness)**.** *If V accepts journal J for shot x, then except with negligible probability there exist B, s with $H(\mathsf{encode}(B) \| s) = C_P$, $\mathsf{legal}(B)$, and $J.hit, J.sunk, J.def$ equal to the rule functions evaluated at (B, x, F_V).*

Sketch. STARK soundness gives execution of the pinned guest on *some* witness (B, s) producing J; the guest's first assertion forces $H(\mathsf{encode}(B) \| s) = J.C = C_P$; legality of B follows from the setup receipt for C_P and collision resistance (two different boards behind one C collide); the guest computes the rule functions on $(B, x, J.\mathcal{H})$ by construction, and V's check $J.\mathcal{H} = F_V$ substitutes the true history. □

Claim 2 (Board binding across the match)**.** *All accepted journals for commitment C_P are consistent with a single board B, except with negligible probability.*

Sketch. Each accepted journal is backed by a witness hashing to C_P; two witnesses with different boards are a SHA-256 collision (the encoding is injective on legal boards: it lists each ship's kind, anchor, and orientation canonically ordered). □

Claim 3 (Termination soundness and undeniability)**.** *V obtains a transferable proof of victory exactly when $F_V \cup \{x\}$ covers all 17 ship cells of P's committed board; P cannot postpone this by any choice of messages other than refusing to answer.*

Sketch. def is computed in-guest from $(B, x, \mathcal{H})$ with $\mathcal{H}$ bound to F_V; by response soundness it equals the true predicate. A responder wishing to avoid $def = \mathsf{true}$ must present a different history (rejected by V's comparison), a different board (a collision), or no proof at all (an abort, visible as such). Attacks A1–A3 are excluded identically. □

Claim 4 (Secrecy)**.** *A (possibly malicious) verifier interacting with an honest prover learns nothing about the prover's board beyond the game-inherent information: the hit/miss/sunk/defeated values of the queried cells, and what those imply.*

Sketch. The prover's messages are receipts; by the zero-knowledge property of the proof system the verifier's view is simulatable from the journals alone, and journals contain only C (hiding, by the 256-bit salt), the verifier's own shots, and the game-inherent bits. Note this is a *leaky-function* guarantee by design: Battleship's answers themselves narrow the board hypothesis space — that is the game — and the protocol protects exactly the remainder. □

Replay, reflection, reordering. Journals bind $(C, x, \mathcal{H})$; a replayed receipt fails the x or $\mathcal{H}$ check, and receipts from the mirrored direction reference the wrong commitment. A commitment-mirroring opponent is rejected at setup (and could not answer shots regardless).

Malicious relay. The relay carries commitments, coin values, shot coordinates, and receipts — all public. It cannot alter any of them undetected (verification fails closed), cannot forge outcomes (it cannot produce receipts for the pinned image IDs), and learns nothing the transcript does not already reveal. Its full power is denial of service.

Prover-side side channels. Proof *contents* leak nothing beyond journals, but proving *time* is observable by the opponent. If guest control flow depended on ship positions, latency could correlate with the secret. All board-touching code in the shared crate is therefore written with fixed-shape loops (a legal fleet always occupies exactly 17 cells) and short-circuit-free boolean accumulation, targeting an instruction trace whose length is independent of the witness; we verify cycle-count invariance empirically in Section 8. We do not extend this claim to physical side channels on the prover's own machine, which are out of scope.

7 Implementation

The system is ∼3,400 lines of Rust and TypeScript, open source under MIT:

- `core` — the shared rules crate described above: types, fleet legality, hit/sunk/defeat, canonical encoding. Compiled unchanged into both the native host and the RISC-V guest, so the rules that are *proven* are bit-for-bit the rules the application *plays*. Unit tests include an equivalence suite pitting the fixed-shape implementations against naive references over random boards.
- `methods/guest` — the two guest programs (commit, shot), ∼40 lines each on top of `core`; SHA-256 uses the zkVM's accelerated implementation.
- `host` — prover/verifier library and the *prover agent*: a localhost-only HTTP service that holds the player's board and salt, proves the player's own answers, and verifies the opponent's (receipt, image ID, commitment, shot, and history). The build enables RISC Zero's `disable-dev-mode`; combined with scrubbing `RISC0_DEV_MODE` from the environment, mock receipts are impossible rather than discouraged. Adversarial integration tests execute every attack from Section 4 against the real prover and assert failure.
- `relay` — a ∼200-line WebSocket forwarder, with no game logic and no RISC Zero dependency, deployable as a container anywhere; it pairs two connections by room code and shovels bytes.
- `web` — a Next.js interface per player (fleet placement, firing, status), which talks only to that player's own agent and to the relay. The Blum coin flip runs here via WebCrypto, in the trust domain of the player it protects.

Two deployment properties are worth noting. First, secrets have a one-process blast radius: board and salt exist only inside the player's own agent, which binds to `127.0.0.1` and answers CORS only for the local UI origin. Second, both agents display their guest image IDs (`/health`); players can compare them out of band, though a mismatch is in any case unexploitable — it merely makes every cross-verification fail.

8 Evaluation

Setup. All measurements are CPU-only on a consumer laptop: Intel Core Ultra 7 155H (22 hardware threads), 15 GB RAM available to the measurement environment (WSL2, Ubuntu 22.04), `rustc` 1.96, `risc0-zkvm` 3.0.5, default composite receipts and `ProverOpts::succinct()` where indicated. Wire size is the JSON-serialized receipt as actually sent through the relay.

Latency and playability. Setup costs one commit proof per player (21 s, produced concurrently by the two machines) plus a sub-millisecond coin flip. Each turn then costs one shot

Proof	Receipt	$\lvert\mathcal{H}\rvert$	Prove (s)	Verify (ms)	Journal (B)	Seal (KiB)	Wire (KiB)
commit	composite	–	21.0	34.5	128	215.7	554.1
shot	composite	0	30.0	34.3	152	215.7	554.1
shot	composite	25	47.2	27.1	352	238.2	613.6
shot	composite	50	41.1	28.2	552	238.2	614.4
shot	composite	99	42.3	32.8	944	238.2	616.1
commit	succinct	–	54.3	31.1	128	217.4	569.5
shot	succinct	0	54.8	29.0	152	217.4	569.5
shot	succinct	99	76.8	27.1	944	217.4	572.8

Table 1: Proving and verification cost (single runs; proving uses all 22 threads). $\lvert\mathcal{H}\rvert$ is the length of the bound shot history. Wire size is the JSON-serialized receipt as sent through the relay; Seal is the raw STARK seal within it.

proof (30–47 s) plus verification (≈30 ms) and relay round-trips. Guest execution grows linearly with the bound history — 20.3k user cycles at $\lvert\mathcal{H}\rvert = 0$ versus 101.8k at $\lvert\mathcal{H}\rvert = 99$ — but the prover pads execution into power-of-two–sized segments, so proving cost moves only at segment boundaries: an empty history fits one 2^{16}-cycle segment, and all longer histories in a maximal game fit two. History binding is therefore close to free in proving time, and its journal overhead is under 1 KB against a ≈600 KiB receipt even at $\lvert\mathcal{H}\rvert = 99$; the $O(n^2)$ transcript overhead discussed in Section 5 is immaterial. A typical match (≈60–90 shots in total) spends 30–60 minutes of per-move proving spread over the game — the pace of a relaxed correspondence game; every individual wait stays under a minute on this hardware, and drops with core count, GPUs, or newer provers [11].

Succinct receipts compress a whole execution to a constant-size seal (217.4 KiB regardless of history) at 1.4–1.8× the proving time. At this circuit size a composite receipt spans only one or two segments, so succinctness buys nothing on the wire; we default to composite receipts and note that recursion becomes worthwhile only for aggregating many moves (Section 9).

Witness independence. Executing the shot guest for a fixed shot and 30-shot history across 20 random boards yields execution lengths between 45,147 and 45,173 cycles — a residual spread of 26 cycles ($< 0.06\%$) from runtime bookkeeping outside the rule functions. The spread vanishes in proving: all 20 executions pad to the identical 2^{16}-cycle segment shape, so proving time carries no signal about the board. Together with the fixed-shape rule implementations this substantiates the side-channel claim of Section 6 empirically rather than by inspection alone.

Comparison with on-chain designs. A direct quantitative comparison is not meaningful (different trust models and hardware), but the qualitative trade-off is: on-chain systems pay consensus latency (tens of seconds per move on proof-of-work Ethereum in [13]) and per-move transaction fees, and expose game metadata publicly forever; our design pays only local proving time and relay round-trips, keeps the transcript private to the players (who may publish it for audit), and gives up the chain's built-in remedy for aborts (stake slashing), which we discuss next.

9 Limitations and scope

Aborts. No zero-knowledge protocol can force a losing player to keep producing proofs; a player facing defeat can disconnect. What our protocol guarantees is that an abort is *attributable and unfalsifiable* — the transcript shows exactly who stopped answering and that every answered shot was honest — and that a completed game's outcome is beyond dispute. Remedies (forfeit

timers socially, or stakes with slashing if one *chooses* to add a chain or an escrow) are orthogonal and composable with the protocol as-is.

Trusted rulebook. Soundness is relative to the correctness of the ∼300-line rules crate and the zkVM toolchain. The crate is small, shared verbatim between play and proof, and tested adversarially; the toolchain trust is shared with the entire zkVM ecosystem [10].

Metadata. The protocol hides boards, not behaviour: timing, move choices, and disconnects are visible to the opponent and relay, and proving-time side channels are mitigated (fixed-shape traces), not physically eliminated. Reusing a board or salt across games voids the hiding analysis; the implementation generates fresh salts per game and warns against board reuse.

Generality. The construction generalises directly to any two-player, turn-based, hidden-state game whose public state is a deterministic function of the (public) move sequence — e.g. Stratego-like reveals or fog-of-war variants — by replacing the rule functions inside the guest. Games with hidden *moves*, simultaneous turns, or more than two players reintroduce the need for a shared ordering or fairness layer and are out of scope.

10 Conclusion

Deployed zero-knowledge Battleship systems delegate the public half of the game — history, turns, termination — to a blockchain. We showed that deleting the chain naively breaks exactly that half (unverifiable sunk reports; the immortal fleet), and that for two-player turn-based games the fix requires no consensus at all: the verifier already owns the canonical history, and binding it into each proof's public output — together with computing the win condition inside the proof — yields self-contained receipts, auditably terminating matches, and a deployment consisting of nothing but the two players and a dumb pipe. The implementation is practical on laptop CPUs and open source; we hope it serves as a minimal, fully-worked pattern for trustless hidden-state games off-chain.

Artifact

Source code (protocol, agent, relay, UI, benchmarks, and the LaTeX for this paper) is available at `https://github.com/hadigghazi/Ledger-Free-Trustless-Battleship`, and the v1.0.0 release is archived at DOI 10.5281/zenodo.21207531. All reported numbers are reproducible via `cargo run -release -p host -bin zk-battleship-bench` and the `-ignored` integration test suite.

References

[1] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.

[2] A. Shamir, R. L. Rivest, and L. M. Adleman. Mental poker. In *The Mathematical Gardner*, pages 37–43. Springer, 1981.

[3] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game, or a completeness theorem for protocols with honest majority. In *Proc. 19th ACM STOC*, pages 218–229, 1987.

[4] M. Blum. Coin flipping by telephone: a protocol for solving impossible problems. *ACM SIGACT News*, 15(1):23–27, 1983.

[5] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018.

[6] J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, LNCS 9666, pages 305–326. Springer, 2016.

[7] J. Bruestle and P. Gafni. RISC Zero zkVM: scalable, transparent arguments of RISC-V integrity. RISC Zero, 2023. https://dev.risczero.com/proof-system-in-detail.pdf.

[8] Succinct Labs. SP1: a performant, open-source zkVM. 2024. https://github.com/succinctlabs/sp1.

[9] A. Arun, S. Setty, and J. Thaler. Jolt: SNARKs for virtual machines via lookups. In *EUROCRYPT 2024*, LNCS 14656, pages 3–33. Springer, 2024.

[10] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari. A survey on the applications of zero-knowledge proofs. arXiv:2408.00243, 2024.

[11] T. Gassmann, S. Chaliasos, T. Sotiropoulos, and Z. Su. Evaluating compiler optimization impacts on zkVM performance. In *Proc. ASPLOS 2026*. ACM, 2026. arXiv:2508.17518.

[12] BattleZips. BattleZips-Circom: zero-knowledge arithmetic circuits for on-chain Battleship. 2022. https://github.com/BattleZips/BattleZips-Circom.

[13] A. Gupta, N. Kaashoek, B. Wang, and J. Zhao. Zero-knowledge Battleship. MIT 6.857 course project report, 2020. https://courses.csail.mit.edu/6.857/2020/projects/13-Gupta-Kaashoek-Wang-Zhao.pdf.

[14] RISC Zero. Battleship example on NEAR. 2022. https://github.com/risc0/battleship-example.

[15] Aleo. ZK Battleship in Leo (workshop example). 2023. https://github.com/AleoNet/workshop.

[16] Mina Foundation. Battleships: a zero-knowledge game on Mina with o1js. 2023. https://github.com/o1-labs.

[17] Dark Forest team. Announcing Dark Forest. 2020. https://blog.zkga.me/announcing-darkforest.

[18] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. Hawk: the blockchain model of cryptography and privacy-preserving smart contracts. In *IEEE S&P 2016*, pages 839–858, 2016.

[19] J. Coleman, L. Horne, and L. Xuanji. Counterfactual: generalized state channels. 2018. https://l4.ventures/papers/statechannels.pdf.